COLLEGE INTERNSHIP MANAGEMENT SYSTEM

A project report
submitted in fulfillment of
the requirements for the award of the degree of

MASTER OF COMPUTER APPLICATIONS

From

APJ Abdul Kalam Kerala Technological University

Submitted By

HISANA HAMZA (MEA25MCA-2010)

MEA Engineering College
Department of Computer Applications
Vengoor P.O, Perinthalmanna, Malappuram, Kerala-679325
OCTOBER 2026

DEPARTMENT OF COMPUTER APPLICATIONS

MEA ENGINEERING COLLEGE
PERINTHALMANNA-679325

CERTIFICATE

*This is to certify that the Project report entitled "**COLLEGE INTERNSHIP MANAGEMENT SYSTEM**" is a bonafide record of the work done by* **HISANA HAMZA (MEA25MCA-2010)** *under our supervision and guidance. The report has been submitted in fulfillment of the requirement for award of the Degree of* **Master of Computer Applications** *from the APJ Abdul Kalam Kerala Technological University for the year 2026.*

Mr. Adil Narakkoden
Assistant Professor
Project Guide
Dept.of Computer Applications

Mr. Sajeesh M
Assistant .Professor
Head of the Department
Dept.of Computer Applications

Acknowledgements

First and foremost, I would like to thank the Almighty God for giving me the knowledge and strength which helped me in the successful completion of this project.

An endeavor over a long period may be successful only with advice and guidance of many well wishers. I take this opportunity to express my gratitude to all who encouraged me to complete this project.I would like to express my deep sense of gratitude to our resepected **Principal Dr.Shahir V.K** for his inspiration and for creating an atmosphere in the college to do the project.

I would like to thank **Mr. Sajeesh M, Assistant Professor and Head of the Department,** for providing permission and facilities to conduct the project in a systematic way. I a m highly indebted to **Mr. Adil Narakkoden**, Assistant Professor in Department of Computer Applications for guiding me and giving timely advices, suggestions and whole hearted moral support in the successful completion of this project.

My sincere thanks to Project Co-ordinator **Mr. Adil Narakkoden,** Assistant Professor in Department of Computer Applications for his wholehearted moral support in completion of this project.

Hisana Hamza (MEA25MCA-2010) DATE: OCTOBER 27, 2026

ABSTRACT

The search for internships and candidate recruitment in educational institutions often involves manual coordination, paper-based resumes, physical notice boards, and fragmented communication, making the overall process time-consuming and difficult to manage. Existing practices lack a centralized platform to verify student academic credentials, monitor available internship opportunities, and track application progress efficiently. As a result, students frequently face delays and lack visibility into their application status, recruiting organizations struggle to filter qualified applicants, and college placement officers face challenges in maintaining transparency and tracking student career outcomes.

To overcome these challenges, I developed **InternHub — College Internship Management System**, a secure web-based application designed for MEA Engineering College that digitizes the complete internship lifecycle. The system enables students to register with their institutional registration numbers and degree programs, upload resumes, browse verified internship opportunities across multiple industries, and submit free applications with a single click. Company recruiters can register their corporate profiles, post internship openings with detailed requirements and stipends, review verified student profiles and CVs, and manage candidates across hiring stages such as Under Review, Shortlisted, Selected, and Rejected. College placement administrators are responsible for managing student records, monitoring employer listings, supervising recruitment activity, and ensuring institutional compliance.

The application provides a centralized platform that improves transparency, eliminates manual paperwork, minimizes processing delays, and streamlines campus placement coordination. It also enables users to track application statuses, view real-time analytics metrics, and receive instant in-app notifications through a modern, responsive user interface. The system was developed using **HTML, CSS, and JavaScript** for the frontend, **Python with Django** for the backend, and **MySQL** as the relational database to provide secure, reliable, and efficient management of college internships and placement operations.

CONTENTS

1. INTRODUCTION **1**

2. SYSTEM ANALYSIS **3**

2.1 Existing System 4

2.2 Proposed System 4

2.3 Module Description 4

2.4 Sprint 5

3. FEASIBILITY STUDY **7**

3.1 Economical Feasibility 8

3.2 Technical Feasibility 8

3.3 Operational Feasibility 8

3.4 Behavioural Feasibility 9

3.5 Software Feasibility 9

3.6 Hardware Feasibility 9

4. SOFTWARE ENGINEERING PARADIGM **10**

4.1 Agile Model 11

4.2 Scrum 11

5. SYSTEM REQUIREMENT SPECIFICATION **12**

5.1 Software Requirements 13

5.2 Hardware Requirements 13

6. SYSTEM DESIGN **14**

6.1 Database Design 15

6.2 Tables 16

6.3 UML Design 18

6.4 Use Case Diagram 18

6.5 Scenario 20

6.6 Data flow diagram 21

7. SYSTEM DEVELOPMENT **25**

7.1 Coding 26

8. SYSTEM TESTING AND IMPLEMENTATION **29**

8.1 Types of Testing 30

8.2 Implementation 31

9. SYSTEM MAINTENANCE **32**

10. FUTURE ENHANCEMENT **34**

11. CONCLUSION **36**

12. APPENDIX **38**

13. BIBLIOGRAPHY **47**

INTRODUCTION

1. INTRODUCTION

Facilitating career development and bridging the gap between academic education and industry exposure require structured coordination, verified credentials, and continuous tracking between students, educational institutions, and recruiting organizations. However, many academic institutions still depend on traditional manual procedures such as physical notice boards, paper-based resumes, email exchanges, and offline spreadsheets for circulating internship openings, verifying student qualifications, and monitoring recruitment outcomes. These methods are often slow, error-prone, and inefficient, resulting in missed application deadlines, fragmented communication, lack of visibility into student hiring stages, and limited transparency throughout the internship acquisition process.

To address these issues, I developed **InternHub — College Internship Management System**, a secure web-based platform designed to automate and simplify the complete campus internship and recruitment workflow for MEA Engineering College. The system provides a centralized environment where students can register their accounts, verify their official college registration numbers, select their enrolled engineering departments, and upload their digital resumes. Using this standardized institutional profile, students can browse verified company internship openings, review prerequisites, stipends, and deadlines, and submit applications directly and free of charge through the application.

The platform also enables company recruiters to create corporate profiles, publish detailed internship listings, examine verified student applications and CVs, and manage candidates across structured hiring stages—including review, shortlisting, selection, and rejection—while dispatching instant automated notifications to applicants. College administrators and placement officers are responsible for supervising the student directory, overseeing employer postings, tracking placement statistics, and monitoring overall campus recruitment activities. By integrating these operations into a single platform, the system improves transparency, accountability, and operational efficiency while significantly reducing manual effort for both students and placement coordinators.

SYSTEM ANALYSIS

.

.

2. SYSTEM ANALYSIS

2.1 Existing System

In the existing system, internship notifications and student applications are handled manually using notice boards, paper resumes, and offline communication. Students must visit the college placement cell or departmental notice boards to submit printed credentials, check eligibility criteria, and inquire about open positions. Placement coordinators and recruiters manually verify student registration details, track candidates using spreadsheets, and communicate updates through disjointed emails or phone calls, making the process slow, inefficient, and prone to errors. The absence of a centralized system also makes it difficult to monitor active internship listings, application progress, and overall campus recruitment outcomes effectively.

2.2 Proposed System

The proposed College Internship Management System (InternHub) provides a centralized web-based platform for managing the complete campus internship and recruitment process. Students can register with their MEA College registration numbers, upload digital resumes, browse verified company openings, and submit internship applications online free of cost. Company recruiters can post internship vacancies, review student credentials, examine resumes, and shortlist, select, or reject applicants through structured hiring stages. The system improves efficiency, institutional transparency, recruitment coordination, and real-time application status tracking.

2.3 Module description

The system is divided into 3 modules based on user roles, each with specific functionalities:

Admin

- View system dashboard and campus placement statistics
- Manage student directory with MEA registration numbers and departments
- Supervise and manage registered company profiles
- Monitor, review, and manage all posted internship opportunities
- View all registered users and track overall recruitment activities

Company

- Register organization profile and secure login
- Manage company profile, contact information, and website details
- Post and manage internship vacancies with stipend, duration, and deadlines
- View and review student applications submitted for posted openings
- Verify candidate MEA registration numbers and view/download resumes
- Update applicant hiring stages (Pending, Under Review, Shortlisted, Selected, Rejected)

Student

- Register and login
- Manage student profile and upload/update digital resume
- Browse, search, and filter verified internship opportunities
- View detailed internship requirements, location, duration, and stipends
- Submit free internship applications with a single click
- Track live application progress, view dashboard metrics, and receive instant notifications

2.4 Sprint

Sprint 1

Module	Task	Pending task if any	Hour of completion	Expected date of completion	Actual date of completion	Reason for delay
Student	Student Registration & Login page	-	2 hr.	15/07/2026	15/07/2026	-
	Profile & Resume Upload page	-	3 hr.	17/07/2026	17/07/2026	-
	Internship Listings & Search	-	2 hr.	19/07/2026	19/07/2026	-
	Application Submission page	-	3 hr.	21/07/2026	21/07/2026	-
	Application Status Tracking	-	2 hr.	23/07/2026	23/07/2026	-
	Notifications & Alerts page	-	1 hr.	25/07/2026	25/07/2026	-

Sprint 2

Module	Task	Pending task if any	Hour of completion	Expected date of completion	Actual date of completion	Reason for delay
Company	Company Login & Registration page	-	1 hr.	02/08/2026	02/08/2026	-
	Company Profile page	-	2 hr.	05/08/2026	05/08/2026	-
	Post Internship Vacancy page	-	3 hr.	08/08/2026	08/08/2026	-
	Internship Management Dashboard	-	2 hr.	11/08/2026	11/08/2026	-
	Applicant Review & Resume Download page	-	3 hr.	14/08/2026	14/08/2026	-
	Candidate Shortlisting	-	2 hr.	18/08/2026	18/08/2026	-

Sprint 3

Module	Task	Pending task if any	Hour of completion	Expected date of completion	Actual date of completion	Reason for delay
Admin	Admin Login & Authentication page	-	1 hr.	01/09/2026	01/09/2026	-
	Admin Dashboard & Placement Overview	-	2 hr.	04/09/2026	04/09/2026	-
	Company Verification & Approval page	-	3 hr.	07/09/2026	07/09/2026	-
	Student & Recruiter User Management page	-	2 hr.	11/09/2026	11/09/2026	-
	Internship Audit & Content Moderation page	-	2 hr.	14/09/2026	14/09/2026	-
	Placement Reports & Data Export page	-	2 hr.	18/09/2026	18/09/2026	-

FEASIBILITY STUDY

3.FEASIBILITY STUDY

A feasibility study is an important process used to determine whether a project can be successfully completed by considering various factors such as legal, economic, technical, operational, and scheduling requirements. It helps project managers understand the practicality of the proposed project before investing significant time, effort, and resources. By evaluating the possible benefits, challenges, risks, and limitations in advance, a feasibility study provides a clear idea of whether the project is realistic and worth pursuing. This allows better decision-making and reduces the chances of unexpected problems during the development and implementation of the project.

3.1 Economical Feasibility

The proposed project is economically feasible because it makes use of open-source technologies and development tools, which significantly reduces the overall cost of software development and implementation. The system also helps reduce the expenses associated with manual paperwork and administrative activities by automating important processes such as internship postings, candidate registration, resume verification, and application tracking. By reducing processing time and minimizing manual effort, the system allows college placement officers and corporate recruiters to utilize their time and resources more efficiently. Since the project does not require expensive proprietary software licenses or specialized hardware infrastructure, it can be hosted and maintained with minimal financial investment. Therefore, the proposed system offers a practical, efficient, and cost-effective solution for managing college internships and campus placement operations.

3.2 Technical Feasibility

The proposed system is technically feasible as it is developed using widely used and reliable web technologies such as HTML, CSS, JavaScript, Python, Django, and MySQL. These technologies provide the necessary support for developing a secure, efficient, and user-friendly web-based application. Python and Django are used to handle server-side operations, business logic, role-based authentication, and request routing, while MySQL provides a structured and reliable relational database to store and manage student profiles, company listings, and application records. The use of Django's Model-View-Template (MVT) modular architecture makes the system easier to develop, maintain, debug, and update. It also allows new features and functionalities to be added in the future without significantly affecting the existing system. Therefore, the selected technologies and system architecture are suitable for successfully implementing and maintaining the proposed application.

3.3 Operational Feasibility

The proposed system is operationally feasible as it provides a simple, clear, and user-friendly interface for students, company recruiters, and college placement administrators. It streamlines and automates important activities such as internship discovery, application submission, resume verification, and candidate hiring status tracking. This reduces the need for manual paperwork and minimizes errors that may occur during manual processing. Company recruiters and placement administrators can review applications and manage candidate hiring stages more efficiently, while students can easily submit applications and monitor their status in real time. Overall, the system improves operational efficiency, reduces processing time, and makes the overall campus internship and placement process more organized and convenient.

3.4 Behavioral Feasibility

The proposed system is behaviorally feasible (user-acceptance feasible) because it is designed with simplicity and ease of use in mind, allowing students, recruiters, and placement staff to understand and operate the system with minimal training. Clear navigation, simple forms, well-organized interfaces, and real-time status updates make it easier for users to complete their tasks efficiently. The system provides a consistent workflow for discovering internships, submitting applications, reviewing candidate resumes, and tracking hiring stages, reducing confusion and unnecessary communication. The digital workflow also improves communication and coordination between students, company recruiters, and college placement officers by providing timely access to application progress, institutional credentials, and recruitment updates. Therefore, the system is easy to adopt and is expected to be well accepted by its intended users.

3.5 Software Feasibility

The proposed application is developed using widely supported web technologies such as HTML, CSS, JavaScript, Python, Django, and MySQL. Since it is a web-based application, users can access the system through commonly used modern web browsers such as Google Chrome, Microsoft Edge, and Mozilla Firefox. The application is designed to work across major operating systems, including Windows, Linux, and macOS, without requiring any additional software installation on the user's device. This makes the system easily accessible and convenient for different types of users. The use of standard web technologies also ensures better compatibility, easier maintenance, and flexibility for future improvements and deployment.

3.6 Hardware Feasibility

The proposed system is hardware feasible as it does not require any specialized or high-performance hardware for its operation. Users can access the application using a standard desktop computer, laptop, or smartphone with a stable internet connection and a modern web browser. College placement officers and company recruiters can operate the system using their existing computer systems, which avoids the need to purchase additional or specialized equipment. The system has relatively low hardware requirements, making it practical and affordable to implement within college campus environments or standard cloud servers. This also reduces the overall implementation and maintenance costs while ensuring that the system remains accessible to all intended users.

SOFTWARE ENGINEERING PARADIGM

4. SOFTWARE ENGINEERING PARADIGM

The software engineering paradigm, also known as a Software Process Model or Software Development Life Cycle (SDLC) model, is a systematic approach that defines the processes, methods, and tools used to develop software. It provides a structured framework for planning, designing, developing, testing, deploying, and maintaining software. The SDLC describes the entire life cycle of a software system, starting from the initial concept and requirement analysis and continuing until its implementation, maintenance, and eventual retirement. It helps ensure that software is developed systematically, meets user requirements, maintains quality, and is completed within the planned time and budget.

4.1 Agile model

The Agile SDLC Model is a software development model that combines iterative and incremental approaches, focusing on flexibility, customer satisfaction, and the rapid delivery of working software. In this model, the project is divided into small parts, and development is carried out through repeated cycles called iterations. Each iteration usually lasts one to three weeks and includes planning, designing, coding, and testing. At the end of each iteration, a working version of the software is delivered or demonstrated to the customer and stakeholders for feedback.

Agile divides tasks into small, time-boxed periods to develop specific features for each release. Instead of completing the entire project at once, developers build the software step by step, adding new features in every iteration. Customer feedback is collected regularly and used to improve the product in subsequent iterations. Developers and customers communicate continuously to ensure that the software meets user needs. This approach helps the development team adapt quickly to changing requirements, identify and correct errors early, reduce project risks, and improve software quality. Agile also encourages teamwork, collaboration, and continuous improvement throughout the development process. Each new build adds more features to the previous version, and the final build contains all the required features to meet customer expectations. Therefore, Agile is suitable for projects where requirements may change frequently and regular delivery of working software is important.

4.2 Scrum

Scrum is an Agile framework used to manage and develop products, especially software. It follows an iterative and incremental approach in which work is divided into short, fixed periods called **Sprints**. A Sprint usually lasts two weeks and can continue for up to one month. During each Sprint, the team selects tasks from the Product Backlog, which contains the list of required features and improvements, and works to complete them.

Scrum promotes teamwork, self-organization, continuous communication, and flexibility. Team members conduct short daily meetings called **Daily Scrums**, usually lasting 15 minutes, to discuss completed work, planned tasks, and any difficulties. At the end of each Sprint, the completed work is reviewed, feedback is collected, and improvements are planned for the next Sprint. Scrum defines three main roles: the Product Owner, who manages product requirements; the Scrum Master, who supports the Scrum process; and the Developers, who build the product. Scrum helps deliver working software regularly, adapt to changing requirements, identify problems early, and improve product quality and customer satisfaction.

SYSTEM REQUIREMENTS SPECIFICATION

5. SYSTEM REQUIREMENTS SPECIFICATION

5.1 Software Requirements

Operating system	:	Windows 8 or above
Backend	:	Python, Django
Frontend	:	HTML, CSS, JavaScript
Database	:	MySQL
IDE	:	VS Code

5.2 Hardware Requirements

Processor	:	Intel Core i3 or above
RAM	:	Minimum 8GB (recommended 16GB)
Monitor	:	14" LCD or Above
Key Board	:	108 Keys
Mouse	:	Any type of mouse

SYSTEM DESIGN

6. SYSTEM DESIGN

System design is the first in the development phase for many engineered product or system. It may define the process of applying various techniques and principles for the purpose of defining a device, a process or system in sufficient detail to permit its physical realization.

6.1 Database Design

Database design is the process of producing a detailed data model of a database. This logical data model contains all the needed logical and physical design choices and physical storage parameters needed to generate a design in a data definition language, which can then be used to create a database.

In the relational model, tables and views represent data, while in an object database, entities and relationships correspond to object classes. Database design encompasses not only base data structures but also forms and queries within a database management system. The design process involves several steps, primarily focusing on determining relationships between data elements and establishing a logical structure, leading to normalization.

Normalization:

It is the procedure of transforming a relationship into a standard form. This process addresses issues that may arise from data redundancy, which refers to the repetition of data within the database. Additionally, it helps maintain data integrity and resolves potential problems related to insertion, updating, and deletion anomalies.

Decomposing is the process of breaking down relations into several smaller relations to eliminate anomalies and uphold data integrity. To achieve this, normal forms or guidelines for structuring relations are utilized.

Insertion anomaly: Inability to add data to the database due to absence of other data.

Deletion anomaly: Unintended loss of data due to deletion of other data.

Update anomaly: Data inconsistency resulting from data redundancy and partial update.

Normal Forms: These are the rules for structuring relations that eliminate anomalies.

1. First Normal Form (1NF)

A relation is considered to be in first normal form when all values within that relation are atomic for each attribute. This means that no attribute value can consist of a set of values or, as it's often described, a repeating group.

2. Second Normal Form (2NF)

A relation is in second normal form if it is in first normal form and meets at least one of the following rules:

- Primary key is not a composite primary key
- No non key attributes are present
- Every non key attribute is fully functionally dependent on full set of primary keys.

3. Third normal Form (3NF)

A relation is said to be in third normal form if there exist no transitive dependencies.

Transitive dependency occurs when two non-key attributes depend on each other and on the primary key. This normalization principle helps decompose data into multiple tables, ensuring consistent data maintenance.

In Third Normal Form (3NF), the main goal is to remove transitive dependencies, which means that non-key attributes should depend only on the primary key, not on other non-key attributes. This ensures that every piece of data in a table describes only the entity represented by that table. By eliminating transitive dependencies, we avoid redundancy and make updates, insertions, and deletions more efficient and error-free.

Overall, 3NF improves database design by making it cleaner, more flexible, and easier to maintain. It minimizes duplicate data and maintains data consistency across relationships, which is crucial for large and frequently updated databases.

6.2 Tables

User

FIELD NAME	DATA TYPE	DESCRIPTION
user_id	INT	PRIMARY KEY, AUTO_INCREMENT
username	VARCHAR(150)	UNIQUE, NOT NULL
password	VARCHAR(128)	NOT NULL
email	VARCHAR(254)	NOT NULL
is_staff	BOOLEAN	NOT NULL
is_active	BOOLEAN	NOT NULL
date_joined	DATETIME	NOT NULL

Student

FIELD NAME	DATA TYPE	DESCRIPTION
student_id	INT	PRIMARY KEY, AUTO_INCREMENT
user_id	INT	FOREIGN KEY
registration_number	VARCHAR(30)	UNIQUE, NOT NULL
name	VARCHAR(100)	NOT NULL
email	VARCHAR(254)	NOT NULL
phone	VARCHAR(15)	NOT NULL
course	VARCHAR(100)	NOT NULL
resume	VARCHAR(100)	NULL
created_at	DATETIME	NOT NULL

Company

FIELD NAME	DATA TYPE	DESCRIPTION
company_id	INT	PRIMARY KEY, AUTO_INCREMENT
user_id	INT	FOREIGN KEY
company_name	VARCHAR(200)	NOT NULL
email	VARCHAR(254)	NOT NULL
phone	VARCHAR(15)	NOT NULL
address	TEXT	NOT NULL
website	VARCHAR(200)	NULL
description	TEXT	NULL
created_at	DATETIME	NOT NULL

Internship

FIELD NAME	DATA TYPE	DESCRIPTION
internship_id	INT	PRIMARY KEY, AUTO_INCREMENT
company_id	INT	FOREIGN KEY
company_name	VARCHAR(200)	NOT NULL
title	VARCHAR(200)	NOT NULL
description	TEXT	NOT NULL
location	VARCHAR(100)	NOT NULL
duration	VARCHAR(100)	NOT NULL
stipend	VARCHAR(100)	NOT NULL
skills_required	VARCHAR(300)	NOT NULL
application_deadline	DATE	NOT NULL
created_at	DATETIME	NOT NULL

Application

FIELD NAME	DATA TYPE	DESCRIPTION
application_id	INT	PRIMARY KEY, AUTO_INCREMENT
student_id	INT	FOREIGN KEY, NOT NULL
internship_id	INT	FOREIGN KEY, NOT NULL
status	VARCHAR(20)	ENUM ('Pending', 'Under Review', 'Shortlisted', 'Selected', 'Rejected')
applied_at	DATETIME	NOT NULL

Notification

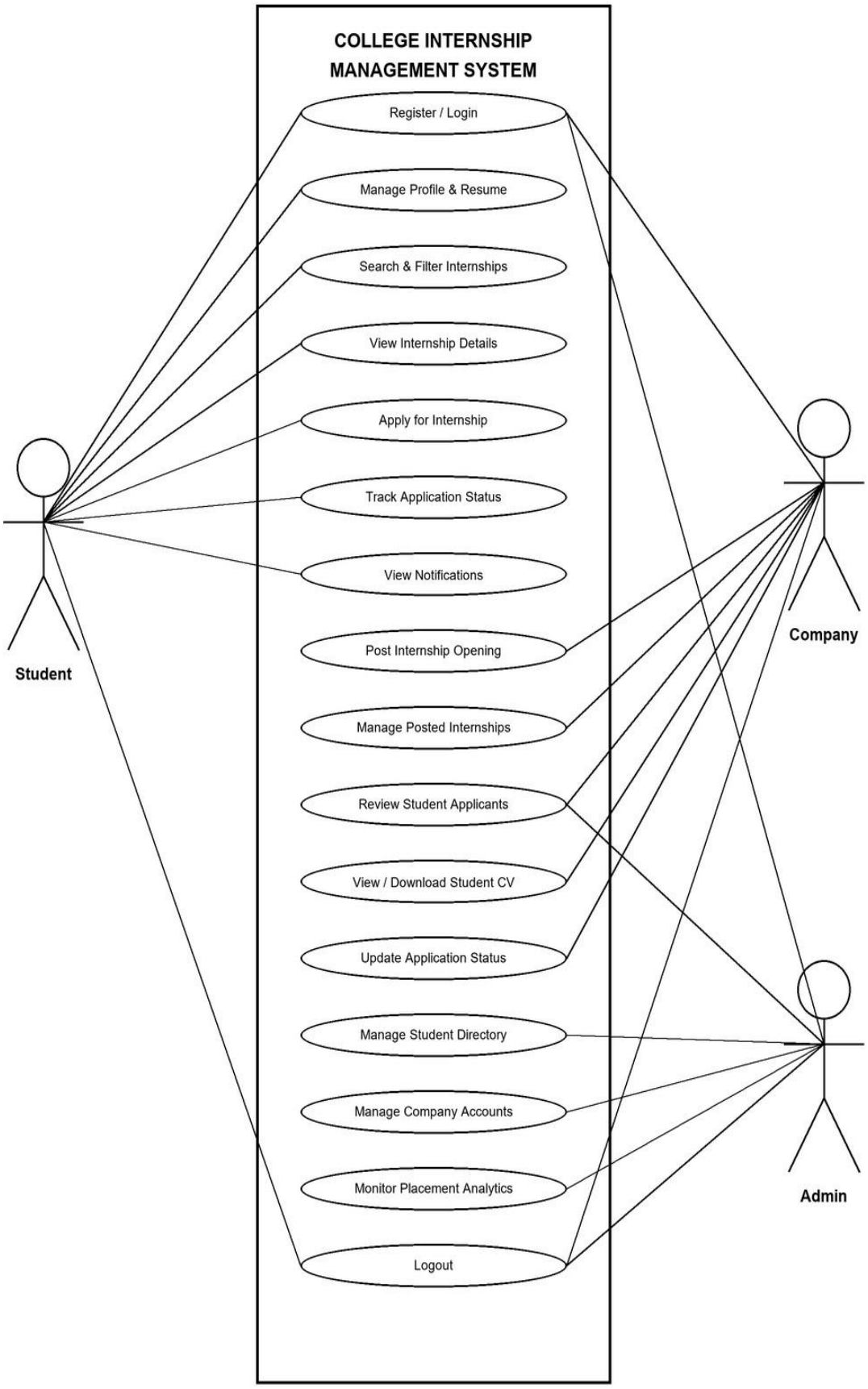
FIELD NAME	DATA TYPE	DESCRIPTION
notification_id	INT	PRIMARY KEY, AUTO_INCREMENT
student_id	INT	FOREIGN KEY, NOT NULL
message	TEXT	NOT NULL
is_read	BOOLEAN	NOT NULL, DEFAULT FALSE
created_at	DATETIME	NOT NULL

6.3 UML Design

The Unified Modeling Language (UML) is a standardized visual language used for specifying, visualizing, designing, constructing, and documenting the various artifacts and components of a software system. It provides a systematic and graphical approach to represent the structure, behavior, functionality, and interactions of a software application, making complex system concepts easier to understand and communicate. UML consists of different types of diagrams that describe both the static and dynamic aspects of a system, such as its users, classes, activities, processes, and interactions between different components. It is particularly important in object-oriented software development because it helps in representing classes, objects, relationships, dependencies, and the overall architecture of the application. In the proposed system, UML diagrams are used to model the major functionalities and interactions between different users and system modules before the actual implementation. These diagrams help developers and stakeholders understand the system requirements clearly, identify the responsibilities of different components, and plan the overall design of the application in an organized manner. UML also helps project teams communicate technical ideas effectively, analyze potential designs, identify possible errors or missing requirements at an early stage, and validate the architectural design of the system. Furthermore, UML diagrams serve as useful documentation throughout the software development life cycle and can assist developers in understanding and maintaining the system when future modifications or enhancements are required. The major UML diagrams used for representing the proposed system include the Use Case Diagram, Class Diagram, Sequence Diagram, and Activity Diagram, with each diagram providing a different view of the system and together giving a complete understanding of its design and functionality.

6.5 Use case diagram

A Use Case Diagram is an essential Unified Modeling Language (UML) diagram that visually illustrates how external users (actors) interact with specific system services (use cases) to achieve defined goals. It models the system's functional requirements from the user's perspective, clearly defining user roles, permissions, and operational boundaries. In the proposed system, it provides a concise overview of key actors and their accessible functionalities. By establishing a shared, intuitive visual representation, the diagram facilitates clear communication among stakeholders, developers, and testers during the analysis and design phases. Ultimately, it helps identify missing requirements, minimizes misunderstandings, and serves as foundational documentation for verifying and maintaining the system throughout its software development life cycle



College Internship Management System

6.5 Scenario

A scenario is a detailed description of a particular situation or sequence of events that helps illustrate how a system or process works. In software engineering, scenarios are often used to show how users interact with a system to achieve specific goals. For example, a scenario might describe how a student registers with their college registration number, browses verified internship openings, submits an application, and tracks their recruitment status, or how an employer reviews candidate resumes and updates hiring stages. Scenarios make it easier to understand user needs, test different conditions, and visualize the flow of operations within a system. They serve as practical examples that bridge the gap between theoretical requirements and real-world usage.

Student

- Student can Register with MEA Registration Number, Department, and Login.
- Student can View and Update Personal and Academic Profile Information.
- Student can Upload, Preview, and Replace Digital Resume / CV (PDF or DOCX).
- Student can Search and Browse Active Internship Postings by Role, Skills, or Company.
- Student can Filter Internships by Location, Duration, and Stipend.
- Student can View Detailed Internship Descriptions, Prerequisites, and Deadlines.
- Student can Submit Free Internship Applications with a Single Click.
- Student can View Application Previews with Registration Number and Resume Status Alerts.
- Student can Track Real-Time Application Hiring Stages (Pending, Under Review, Shortlisted, Selected, Rejected).
- Student can Safely Withdraw Pending Applications Before Employer Review.
- Student can View Dashboard Analytics, Metric Counters, and Application History.
- Student can Receive and View Real-Time In-App Status Notifications.
- Student can Logout.

Company

- Company can Register with Corporate Profile, Contact Information, and Login.
- Company can View and Update Company Profile, Website, and Address Details.
- Company can Post New Internship Opportunities with Title, Location, Stipend, and Deadlines.
- Company can Manage, Edit, and Monitor All Active Internship Postings.
- Company can View Total Applicants Count and Detailed Candidate Submissions for Each Posting.
- Company can Verify Candidate MEA Registration Numbers, Enrolled Departments, and Contact Info.
- Company can View and Download Student Resumes / CVs with a Single Click.
- Company can Update Applicant Hiring Stages (Pending, Under Review, Shortlisted, Selected, Rejected).

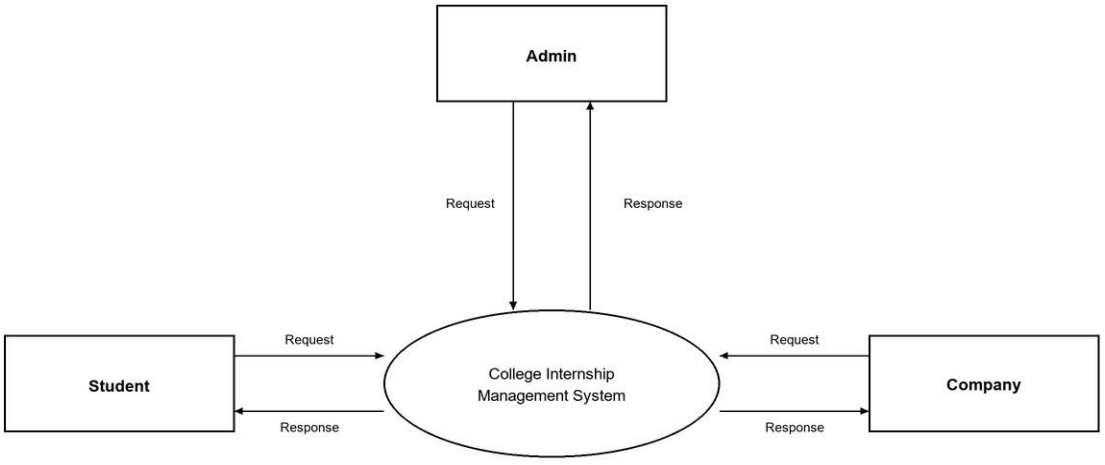
- Company can Automatically Trigger In-App Status Notifications to Candidates upon Stage Updates.
- Company can View Recruiter Dashboard Metrics and Active Hiring Statistics.
- Company can Logout.

Admin

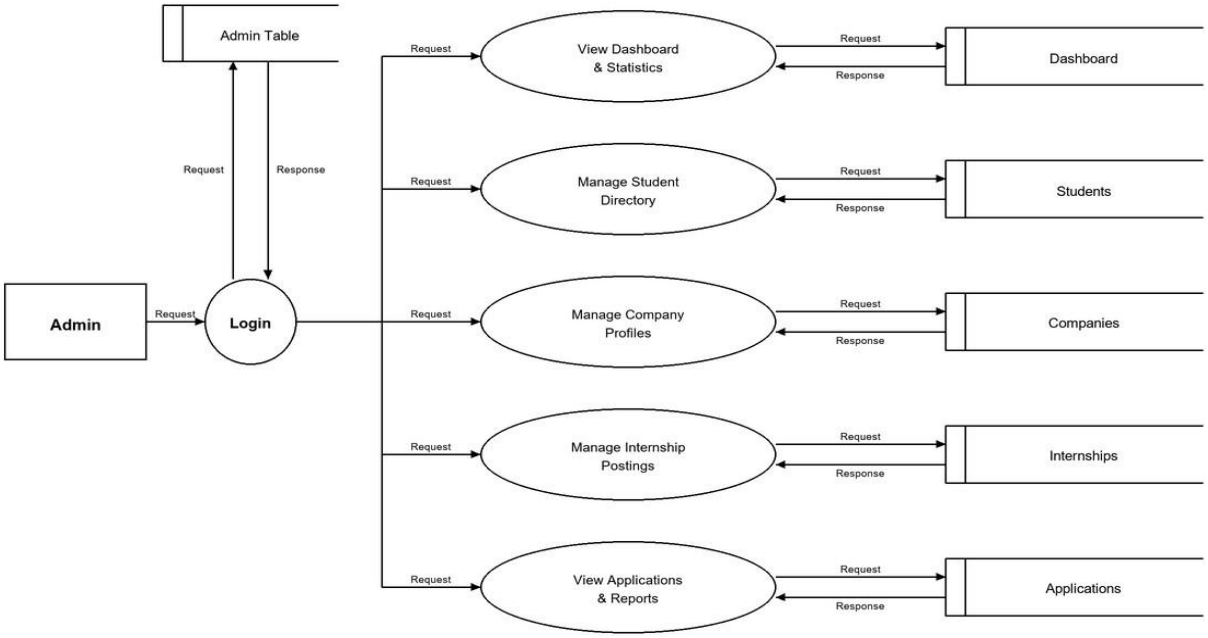
- Admin can Login to the Central Placement Portal and Django Administration Panel.
- Admin can View Central Placement Dashboard, Metric Counters, and Campus Statistics.
- Admin can View, Search, and Filter Registered Student Directory with MEA Registration Numbers.
- Admin can View, Edit, and Manage Student Profiles and Academic Records.
- Admin can Supervise and Manage Registered Company Profiles and Corporate Accounts.
- Admin can Monitor, Review, and Delete Internship Postings across All Organizations.
- Admin can Oversee Student Applications, Candidate Selections, and Placement Records.
- Admin can Manage Database Tables and Enforce Institutional Compliance for MEA Engineering College.
- Admin can Assist with Account Recovery and Administrative Credentials Management.
- Admin can Logout.

6.6 Data Flow Diagram

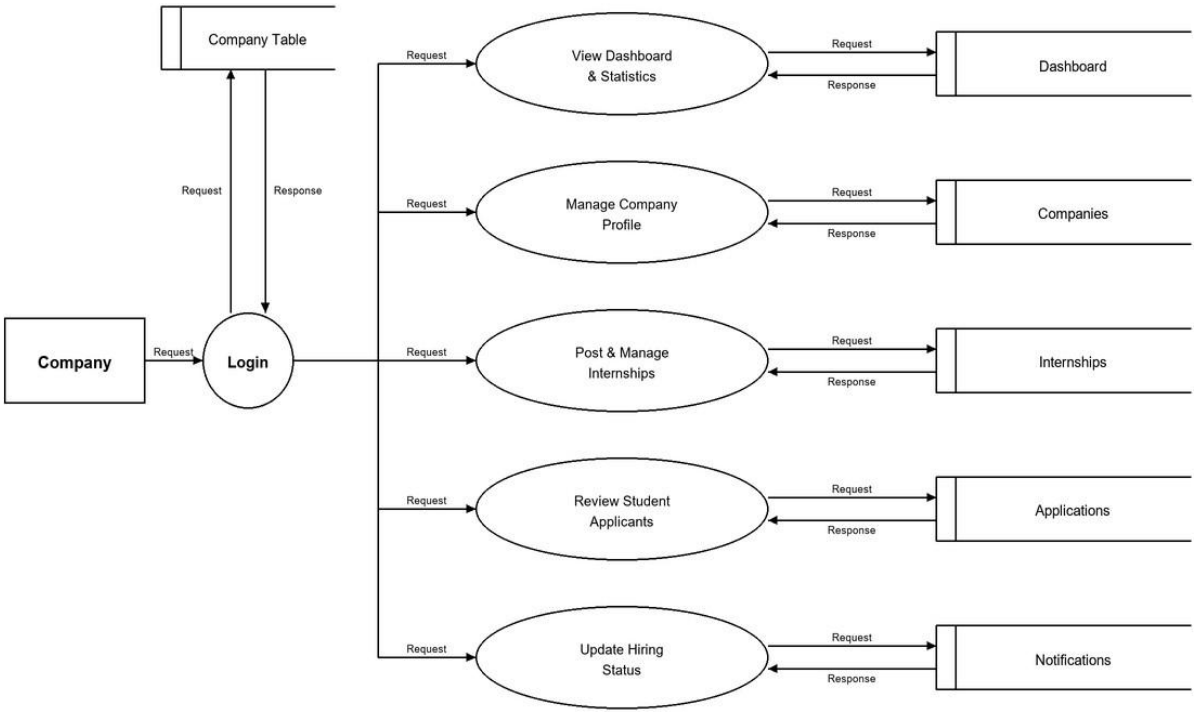
DFD level 0 Diagram



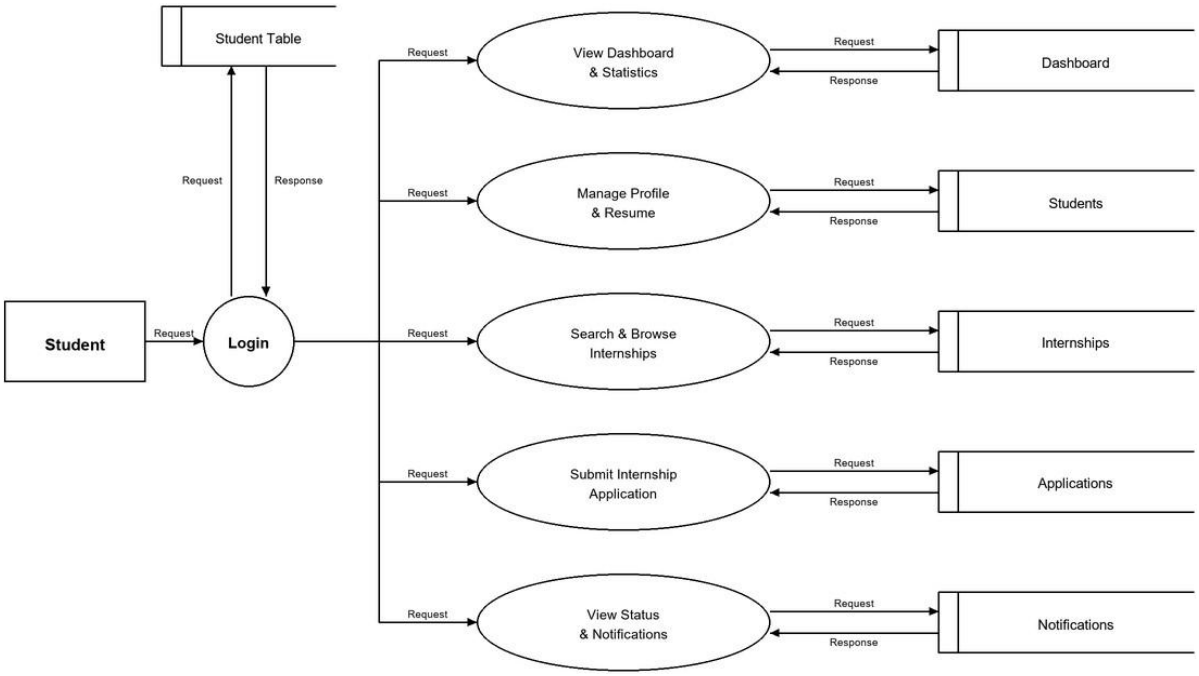
DFD level 1 Admin Diagram



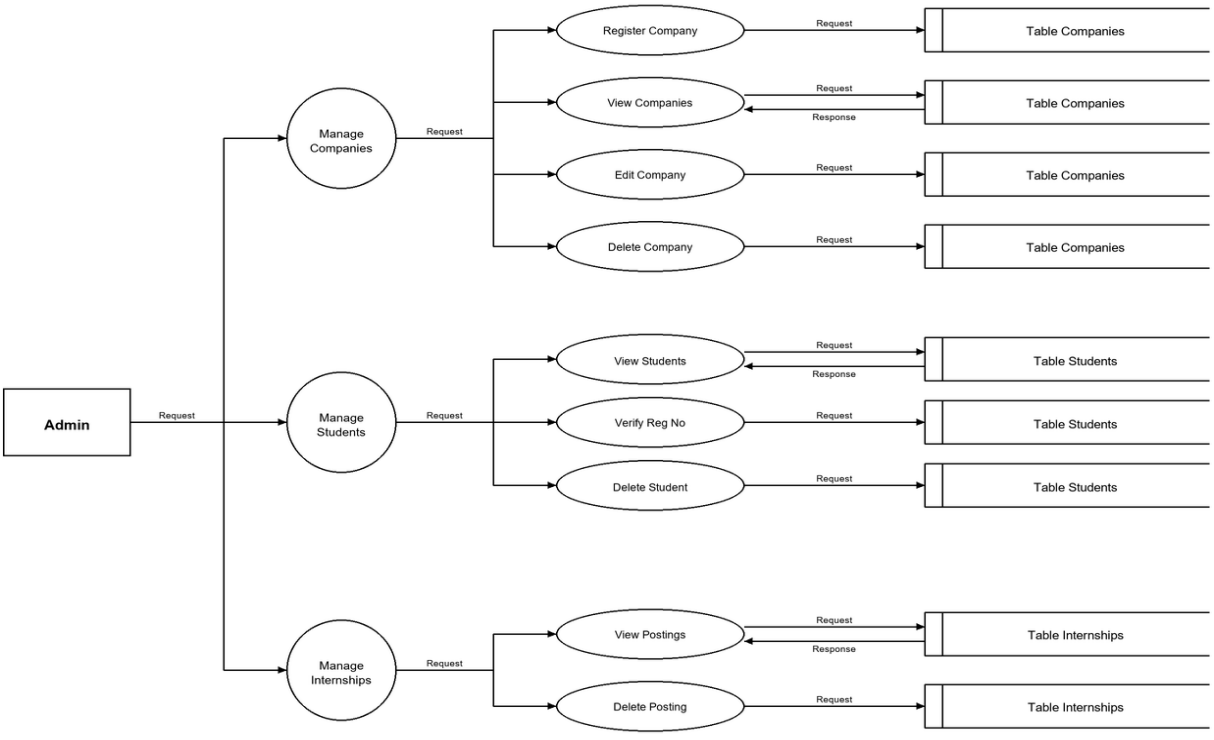
DFD level 1 Company Diagram



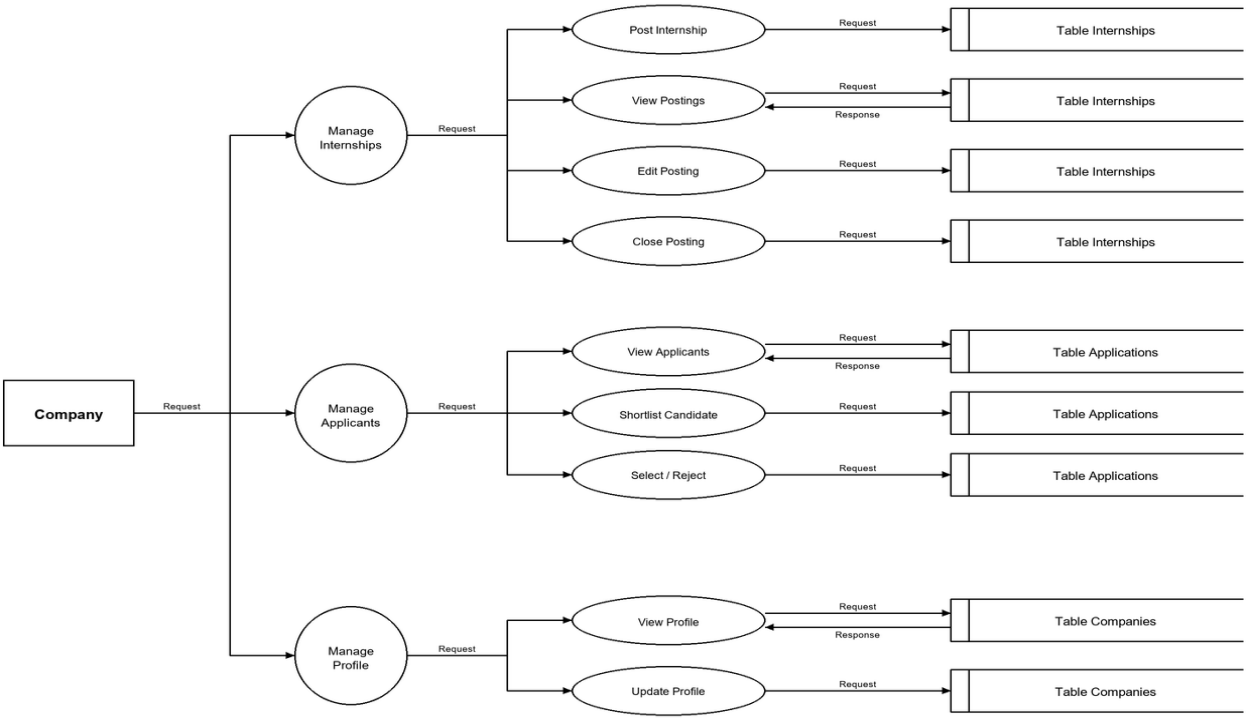
DFD level 1 Student Diagram



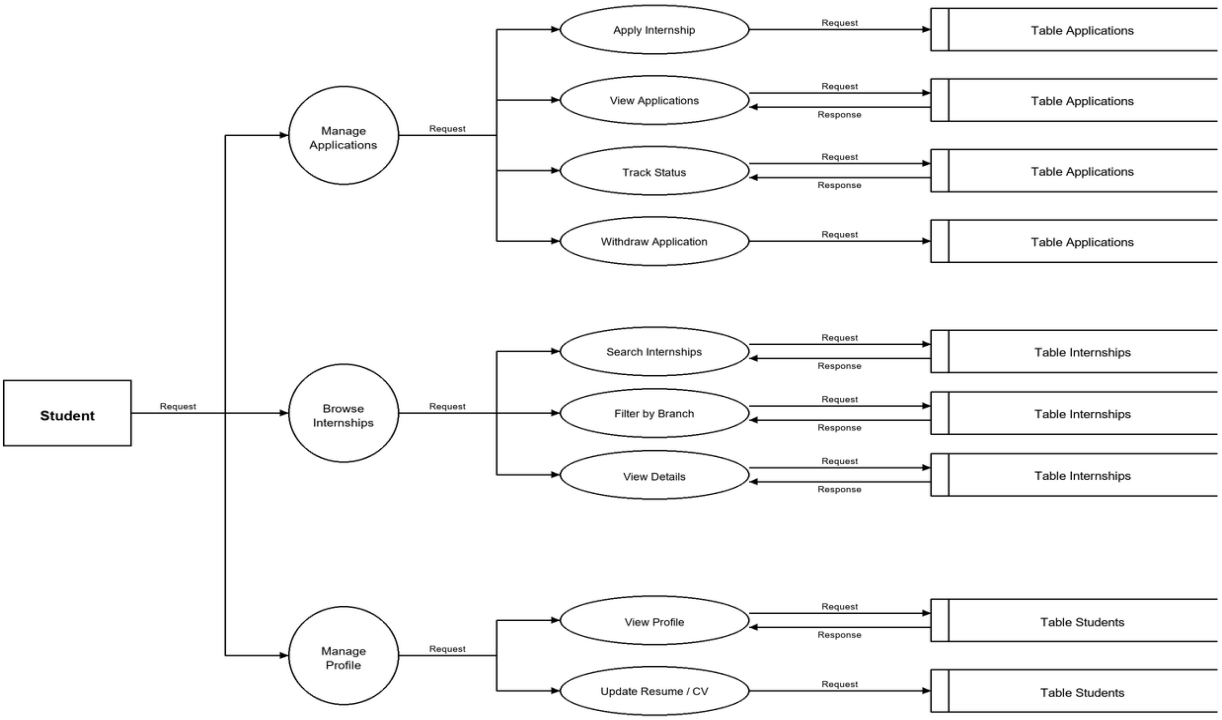
DFD level 2 Admin Diagram



DFD level 2 Company Diagram



DFD level 2 Student Diagram



SYSTEM DEVELOPMENT

7. SYSTEM DEVELOPMENT

System development is a series of operations to manipulate data to produce output from a computer system. The principal activities performed during the development phase can be divided into two major related sequences:

- External system development
- Internal system development

The major external system activities are Implementation Planning, Equipment Acquisition, and Installation. The internal system development activities involve software architecture design, database modeling, API construction, coding, and security integration.

7.1 Coding

The purpose of code is to aid in identifying and retrieving information. It consists of an ordered collection of symbols for unique identification of entities or attributes, illustrating interrelationships among items. Codes facilitate identification, access, sorting, and matching records, ensuring each code has a single, clear meaning for the entity or attribute.

Python

Python is a high-level, interpreted, general-purpose, object-oriented programming language widely recognized for its clean syntax, readability, and versatile standard library. It supports multiple programming paradigms, including procedural, object-oriented, and functional programming, enabling developers to build complex, scalable web backends with minimal boilerplate code. Python features automatic memory management, dynamic typing, and a comprehensive ecosystem of third-party packages, making it an ideal choice for rapid enterprise software development. In the College Internship Management System, Python serves as the foundational programming language executing server-side business logic, managing user authentication workflows, validating candidate academic credentials (such as MEA College registration numbers), and orchestrating database interactions.

Django Web Framework & MVT Architecture

Django is an advanced, open-source, high-level Python web framework designed to follow the "batteries-included" philosophy and the Model-View-Template (MVT) architectural design pattern. It encourages rapid development, clean design, and high maintainability by providing built-in utilities for URL routing, database schema migrations, template rendering, and administrative interfaces:

- **Model:**
  Defines the data structures, relational fields, and database constraints as Python classes using Django's Object-Relational Mapper (ORM), automatically mapping them to underlying MySQL tables.

- **View:**
  Acts as the request-handling controller that processes incoming HTTP GET and POST requests, executes computational and business logic, enforces role permissions, and returns appropriate responses or template renderings.

- **Template:**
  Formats dynamic content using Django's template engine syntax alongside HTML5, CSS3, and JavaScript, presenting an interactive user interface to students, recruiters, and administrators.

SQL & MySQL Database Engine

SQL (Structured Query Language) is the standard programming language used for storing, managing, and manipulating data in relational database systems. It provides standardized commands for data definition (DDL), data manipulation (DML), and data querying (DQL), while guaranteeing data consistency, transaction atomicity, and referential integrity. MySQL is a high-performance, open-source Relational Database Management System (RDBMS) that utilizes SQL to store and manage data in organized, relational tables. It supports primary keys, foreign key constraints, indexing, and multi-user concurrency. In InternHub, MySQL serves as the primary relational database engine, reliably maintaining student records, institutional registration numbers, corporate recruiter accounts, internship listings, submitted applications, and automated system notifications.

Standard Libraries, Built-in Modules & Middleware

The Django and Python development ecosystem utilizes specialized built-in modules and middleware packages to handle security, role verification, file management, and data integrity:

- **Django Authentication & PBKDF2 Password Hashing**:
  Django provides a comprehensive authentication system that handles user login sessions, permission verification, and credential security. Instead of storing plaintext passwords, the system uses the PBKDF2 (Password-Based Key Derivation Function 2) cryptographic algorithm with a SHA-256 hash and a unique salt for each user, providing robust defense against dictionary, rainbow table, and brute-force attacks.
- **Django Object-Relational Mapper (ORM)**:
  The Django ORM abstracts complex SQL queries into Python syntax, allowing safe data manipulation while automatically parameterizing database queries. This eliminates common web vulnerabilities such as SQL injection attacks and enforces referential integrity across related database models.
- **Django File Handling & Media Storage**:
  The system handles `multipart/form-data` uploads for student resumes and curriculum vitae (CVs) in PDF and DOCX formats. Django's file storage backend securely renames, validates, and stores uploaded files within a designated media directory (`media/resumes/`) while recording persistent file references in the database.
- **Security Middleware & CSRF Protection**:
  Django includes a modular middleware pipeline that intercepts incoming requests and outgoing responses. The Cross-Site Request Forgery (CSRF) middleware enforces token-based validation on all state-changing POST forms, preventing unauthorized cross-site command execution. Role-based view decorators (such as `@login_required` and custom `@company_required`) ensure that students cannot access employer recruitment tools, and employers cannot execute student actions.

Application Programming Interface (API)

An Application Programming Interface (API) is a computing interface that defines interactions between multiple software applications or mixed hardware-software implementations. It establishes a standardized set of rules, protocols, and tools that allow different software components to communicate, exchange data, and execute operations seamlessly without exposing underlying implementation mechanics. APIs serve as an abstraction layer between client interfaces and backend servers, facilitating structured data transfer over network protocols such as HTTP/HTTPS.

RESTful Architecture & HTTP Request-Response Lifecycle

Representational State Transfer (REST) is an architectural style designed for distributed networked applications that relies on a stateless, client-server, cacheable communication protocol—primarily HTTP. In this project, client interactions follow a structured HTTP request-response cycle utilizing uniform resource locators (URLs) and standard HTTP protocol methods:

- **GET Requests**:
  Used to retrieve and display resources, such as browsing active internship openings, viewing applicant profiles, and fetching dashboard statistics.
- **POST Requests**:
  Used to transmit secure data payloads to the server, including user authentication credentials, new internship submissions, student application filings, and hiring stage status updates.

A key advantage of this architecture is its clear separation of concerns. Client requests convey the required input parameters along with session cookies and CSRF tokens, enabling the backend Django controllers to process operations, execute database transactions, and return lightweight, updated views with zero cross-session state leakage.

SYSTEM TESTING AND IMPLEMENTATION

8. SYSTEM TESTING AND IMPLEMENTATION

Testing is vital to the success of the system. It makes a logical assumption that if all the parts of the system are correct, the goal will be successfully achieved in this project. It is the stage of implementation, which ensures that system works accurately and effectively before the live operation commences. It is a confirmation that all are correct and opportunity to show users that the system must be tested and show that the system will operate successfully and produce expected results under expected conditions. Software testing is a crucial element of software quality assurance and represents the unlimited review of specification, design and coding. Testing represents an interesting anomaly for the software. During the earlier definition and development phase, it was attempted to build the software from an abstract concept to implement.

Testing involves planned activities conducted systematically to ensure a system operates accurately and efficiently before live operations begin.

8.1 Types of Testing

Different types of testing are:

- Unit testing
- Integration testing
- System testing
- Validation testing
- User acceptance testing.

Unit Testing:

Unit testing typically takes place during a combined code and unit testing phase within the software life cycle, although coding and unit testing may sometimes occur as separate phases. Each module is tested individually upon completion to ensure proper functionality. The focus of unit testing is to evaluate a unit as a whole, assessing the interaction of multiple functions while keeping the test confined to a single unit. This testing is conducted during the programming stage to verify that each module performs satisfactorily against the expected output. Unit Testing is a software testing method where individual components or modules of a program are tested separately to ensure each part works correctly. It helps identify and fix bugs early in development, making the code more reliable and easier to maintain.

Integration Testing:

Integration testing systematically tests the combined modules of a system, such as login, registration, and admin functions, to identify and correct errors at the interface. This process ensures that data integrity is maintained, and that individual modules work together effectively to achieve the desired overall functionality. The goal is to combine unit-tested modules and evaluate them as a complete program.

- The system contains various components and they have to be combined and tested.
- The software is embedded into hardware and tested.
- The modules are combined and tested.
- The hardware part tested separately.

System Testing:

Following the completion of validation testing, the subsequent step involves output testing of the proposed system. A system is deemed ineffective if it fails to generate the required data in the specified format. During output testing, the format of the data displayed or generated by the system is evaluated by soliciting feedback from the user. It has been confirmed that the output format on the screen aligns correctly with the specifications established during the system design phase, tailored to meet user requirements. Consequently, no corrections are necessary for the system as a result of output testing.

User Acceptance Testing:

User acceptance is crucial for a system's success. To ensure this, the system is tested by maintaining communication with prospective users during development, allowing for necessary changes based on their feedback.

- Output Screen design.
- Input Screen design.
- Menu driven system.

Validation Testing

Validation testing is performed to ensure that the developed system meets the specified user requirements and performs the intended functions correctly. It verifies whether the system provides the expected results for different inputs and whether the implemented features satisfy the requirements defined during the system design stage. In this testing, the complete system is tested using realistic inputs and scenarios to confirm that it behaves as expected.For the proposed system, validation testing is carried out by checking important functions such as user registration and login, authorization request processing, laboratory approval, chemical purchase requests, stock reservation, authorization expiry, purchase code generation, and report generation. The actual outputs are compared with the expected outputs to identify any errors or missing functionality.

8.2 Implementation

Implementation is the stage of project, when theoretical design is turned into a working system. The most crucial stage is achieving a successful system and confidence that the new system will work effectively. It involves careful planning, investigation of the manual system and transition to the new system.

Implementation is the process of transforming a new or updated system design into a functional operation, involving all necessary activities to transition from the old manual internship placement methods to the new centralized web-based system.

- Careful planning.
- Investigating the current system and its constraints on implementation.
- Design of methods to achieve the changeover.
- Training of the staff in the procedure and evaluation of change over method.

Implementation is the final and crucial phase of a project. It begins with planning methods for execution. After successful system implementation, user training becomes essential, involving the preparation of user manuals. Changeover is vital, ensuring the developed tool meets requirements before user acceptance. It occurs only when the system satisfies the analysis and all implementation tasks are completed.

SYSTEM MAINTENANCE

9. SYSTEM MAINTENANCE

Maintenance is the process of making adaptations to the software system for external changes (such as new regulatory requirements or feature enhancements) and internal changes (such as fixing bugs and optimizing performance). When changes are implemented during the maintenance phase, all preceding steps of the development lifecycle must be systematically revisited to ensure system stability and data integrity.

There are three types of maintenance:

- Corrective (Fixing bugs/errors)
- Adaptive (Updates due to environment changes)
- Perfective (Enhancements, requirements changes)

Maintenance presents a complex challenge within system development. The definition of software maintenance can be articulated through four key activities that occur after the program is released for use.

Understanding software maintenance is crucial because it's unrealistic to expect that testing will identify every issue in a large system. Ongoing maintenance is necessary due to the rapid changes that occur across all areas of computing. Enhancements often involve recommending new features, modifying existing functions, and making general improvements as the software is utilized. Additionally, changes may be implemented to enhance the software's maintainability or reliability for future use.

9.2 Key Maintenance Activities & Importance

Understanding software maintenance is crucial because it is unrealistic to expect that pre-release testing will identify every potential issue in a multi-user, web-based platform. Ongoing maintenance is necessary due to the rapid technological advancements across web computing, security, and database management.

Within the College Internship Management System (InternHub), the definition of software maintenance is articulated through four key ongoing activities post-release:

1. **Problem Identification & Analysis**: Monitoring Django server logs, HTTP response status codes, database query performance, and user feedback (from students, corporate recruiters, and college placement staff) to isolate root causes of system anomalies or unexpected behaviors.
2. **Design & Code Modification**: Safely modifying Django views, ORM model structures, URL routing rules, or frontend template interfaces without breaking existing user workflows, database schema constraints, or role-based security rules.
3. **Regression Testing & Verification**: Re-executing automated unit test suites (covering user authentication, MEA registration number validation, one-click internship application filing, and recruiter status updates) to ensure bug fixes or feature updates do not introduce unintended side effects.
4. **Deployment & Configuration Management**: Applying database schema migrations (`makemigrations` and `migrate`), maintaining environment variables and secret keys in configuration files, managing media storage for student resumes, and tracking version control history via Git.

FUTURE ENHANCEMENT

10. FUTURE ENHANCEMENT

The College Internship Management System (InternHub) has been designed with flexibility, modularity, and scalability in mind, allowing for future enhancements to further improve its functionality, institutional compliance, and operational efficiency. Some potential areas for future enhancements include:

1. **AI-Powered Resume Parsing & Automated Skill Matching**: Leveraging Natural Language Processing (NLP) and machine learning algorithms to automatically parse uploaded student resumes (PDF/DOCX formats), extract key technical proficiencies, academic projects, and certifications, and calculate compatibility matching scores against active internship requirements to recommend the best-suited candidates to recruiters.

2. **Multi-Channel SMS & WhatsApp Notification Integration**: Implementing external communication APIs and webhooks (such as Twilio or WhatsApp Business API) to deliver real-time, automated alerts regarding application stage updates, shortlisting notices, and interview schedules directly to students' mobile phones, ensuring time-critical placement announcements are never missed.

3. **Digital MoA & Institutional Approval Workflow**: Incorporating an automated No Objection Certificate (NOC) generation pipeline and digital Memorandum of Agreement (MoA) signing mechanisms, enabling college department heads and placement officers to review offer details and digitally sign formal internship authorization letters within the platform.

4. **Integrated Video Interview & Assessment Scheduling**: Connecting the recruiter dashboard directly with WebRTC or video conferencing APIs (such as Zoom or Google Meet API) to allow hiring organizations to schedule, host, and evaluate virtual technical assessments and interview rounds seamlessly without leaving the platform.

By continuously enhancing the College Internship Management System (InternHub) with these innovative features, it can evolve into a comprehensive, highly secure, and intelligent platform for campus recruitment governance, student career advancement, and digital placement management.

CONCLUSION

11. CONCLUSION

The College Internship Management System (InternHub) has been successfully developed with careful planning and implementation to digitize campus internship coordination workflows, improve institutional recruitment oversight, and automate student application processing. The system aims to simplify the posting and discovery of internship opportunities, maintain a centralized repository of verified recruiting companies, and support recruitment lifecycles through real-time application submission, resume management, and status tracking. It also provides role-based access control to ensure that students, company recruiters, and college administrators can perform their respective operations securely and efficiently.

Throughout the development process, rigorous testing, including functional testing of individual modules, role-based security access checks, application deadline validation, and database transaction verification, was conducted to ensure the system's reliability and correctness. The testing results indicate that the system performs its intended functions effectively and helps reduce manual paperwork, minimize administrative delays, and improve transparency in campus recruitment activities. By maintaining comprehensive student profile records, monitoring application statuses, and providing a structured workflow for vacancy publication and candidate evaluation, the system supports a more organized, reliable, and equitable internship selection process.

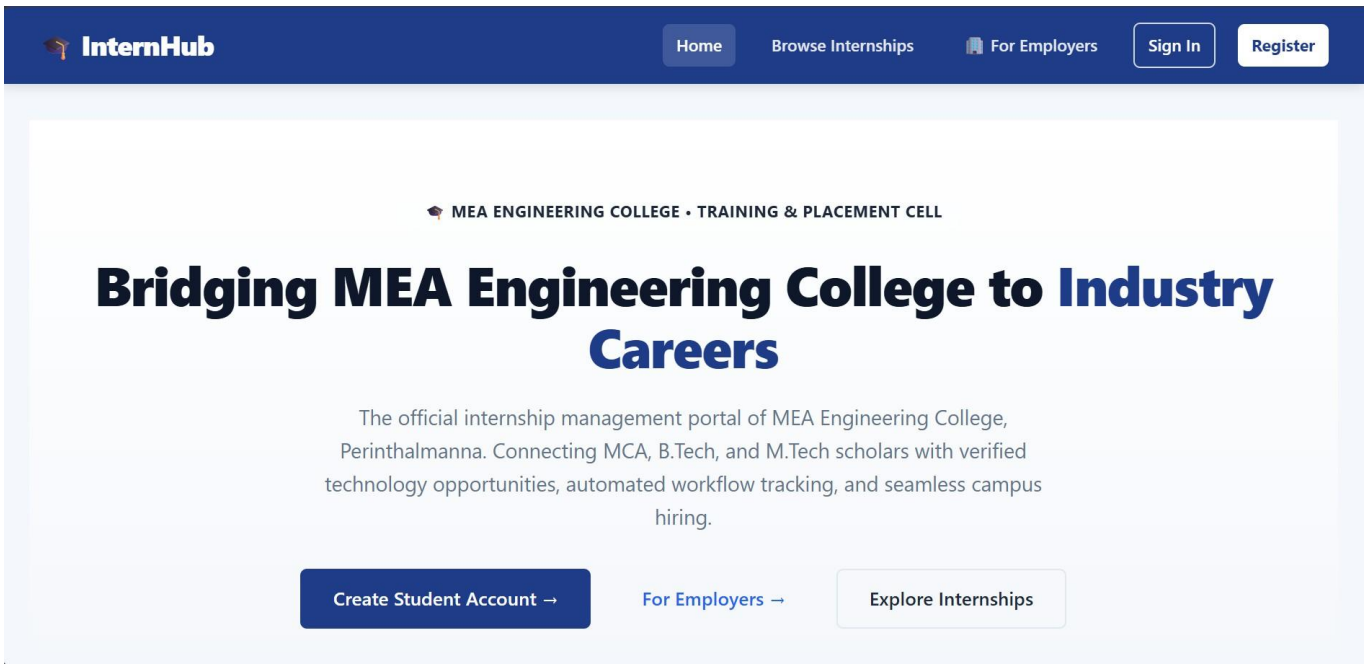
Overall, the project demonstrates how software technology can be used to improve the management of college internships and campus recruitment drives. It provides a reliable and user-friendly platform that addresses the operational needs of student applicants, corporate recruiters, and college placement administrators. The system also offers scope for future enhancements, such as automated resume parsing with AI skill matching, multi-channel instant notifications, and integration with virtual video assessment tools. Thus, the project serves as a practical solution for making college internship administration and placement management more secure, efficient, transparent, and manageable.

.

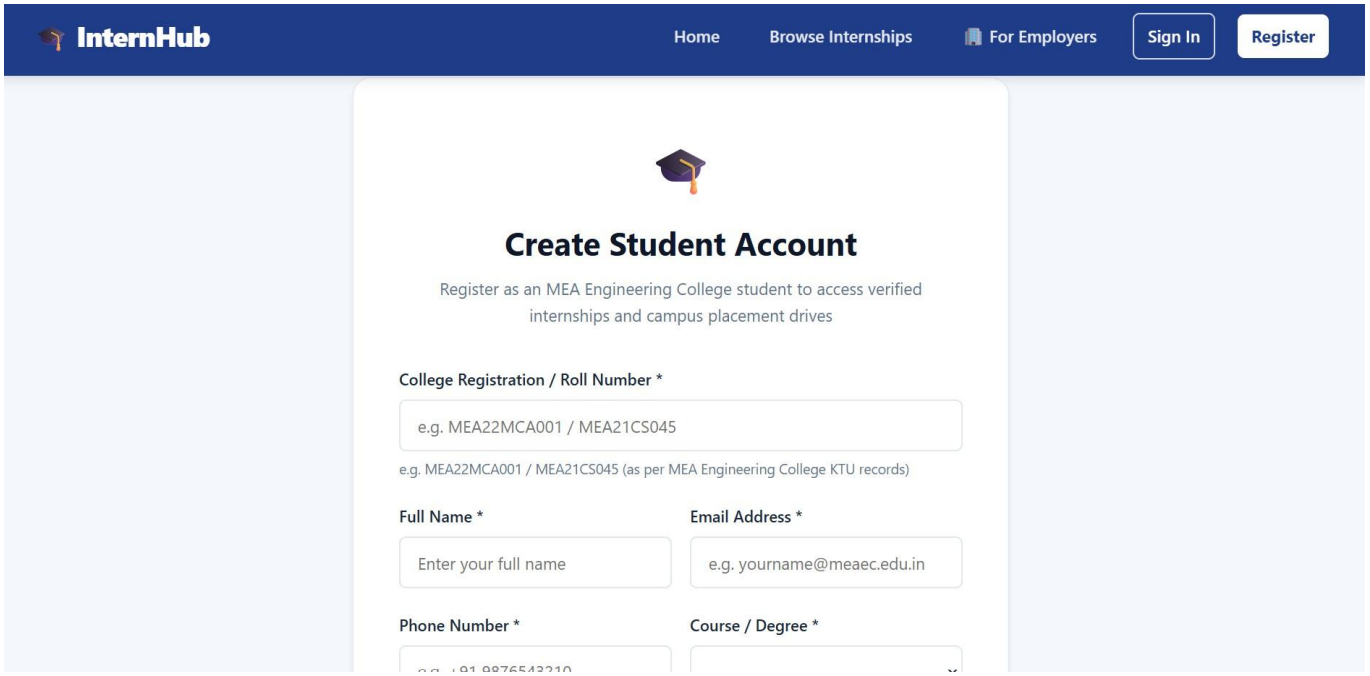
APPENDIX

12. APPENDIX

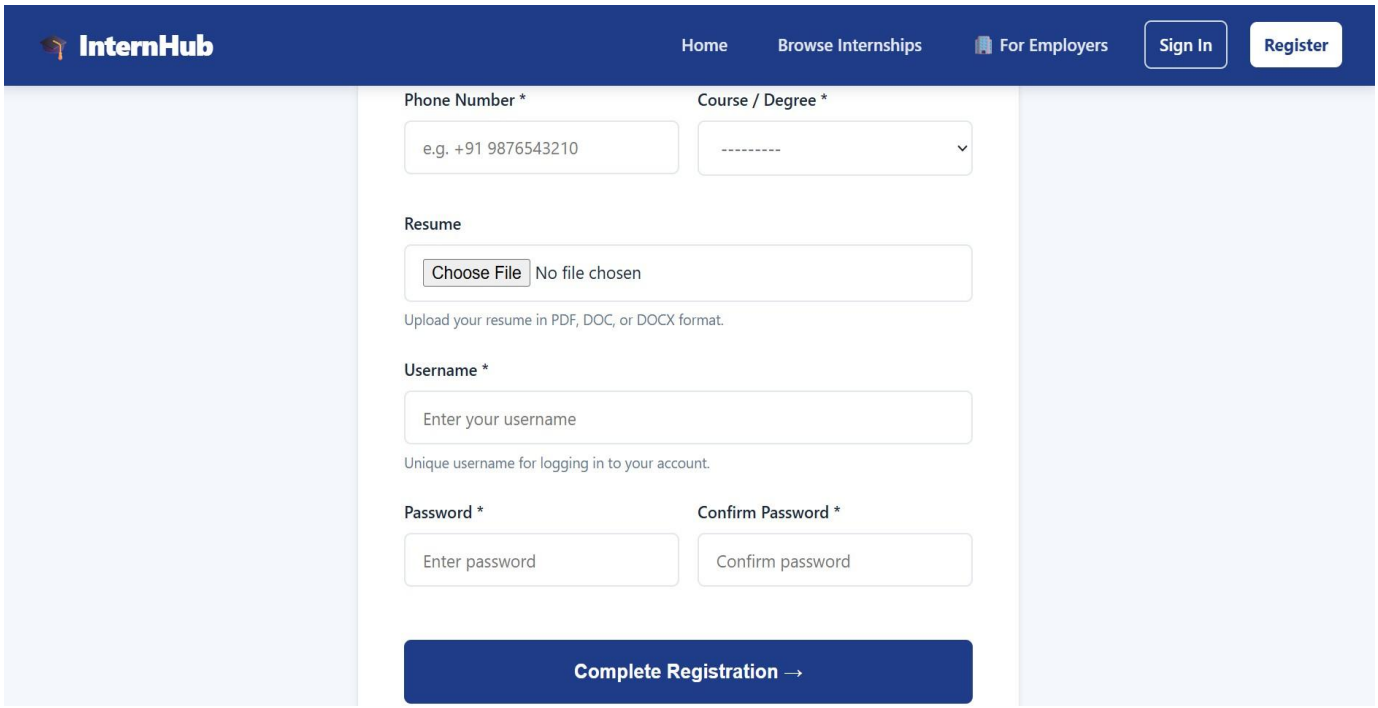
Home page of portal



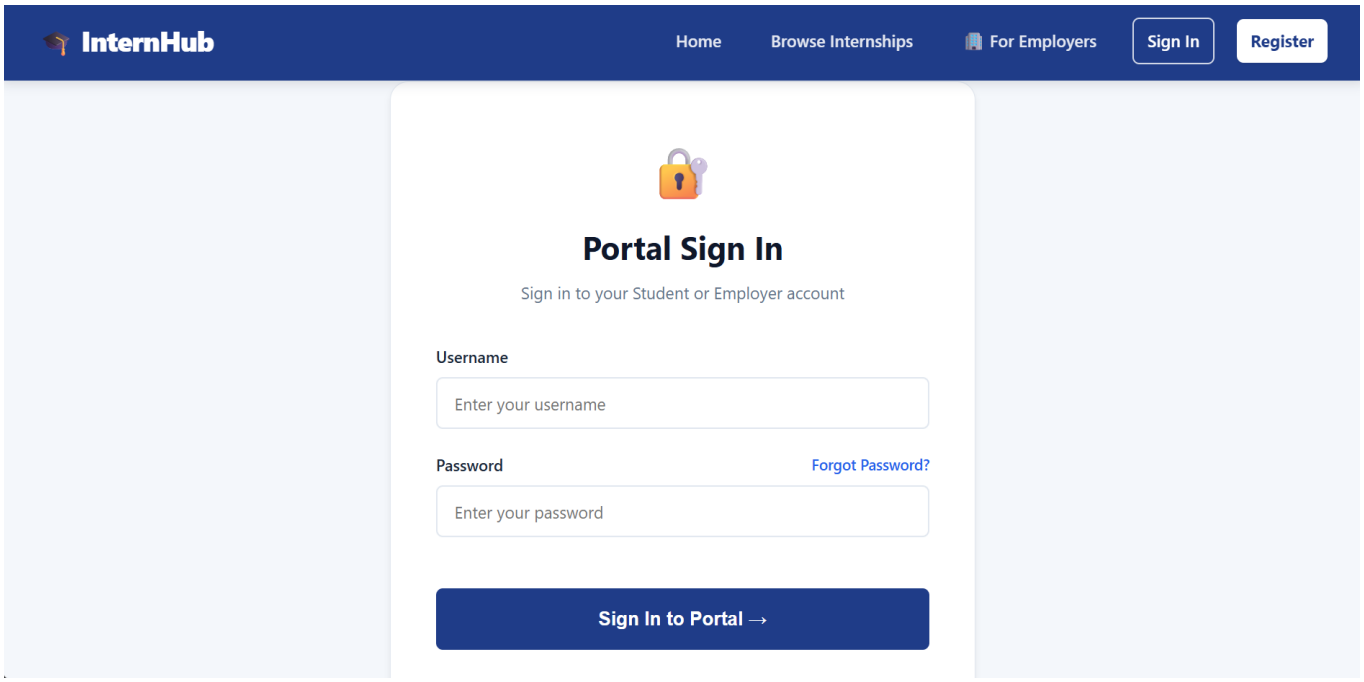
User Registration page



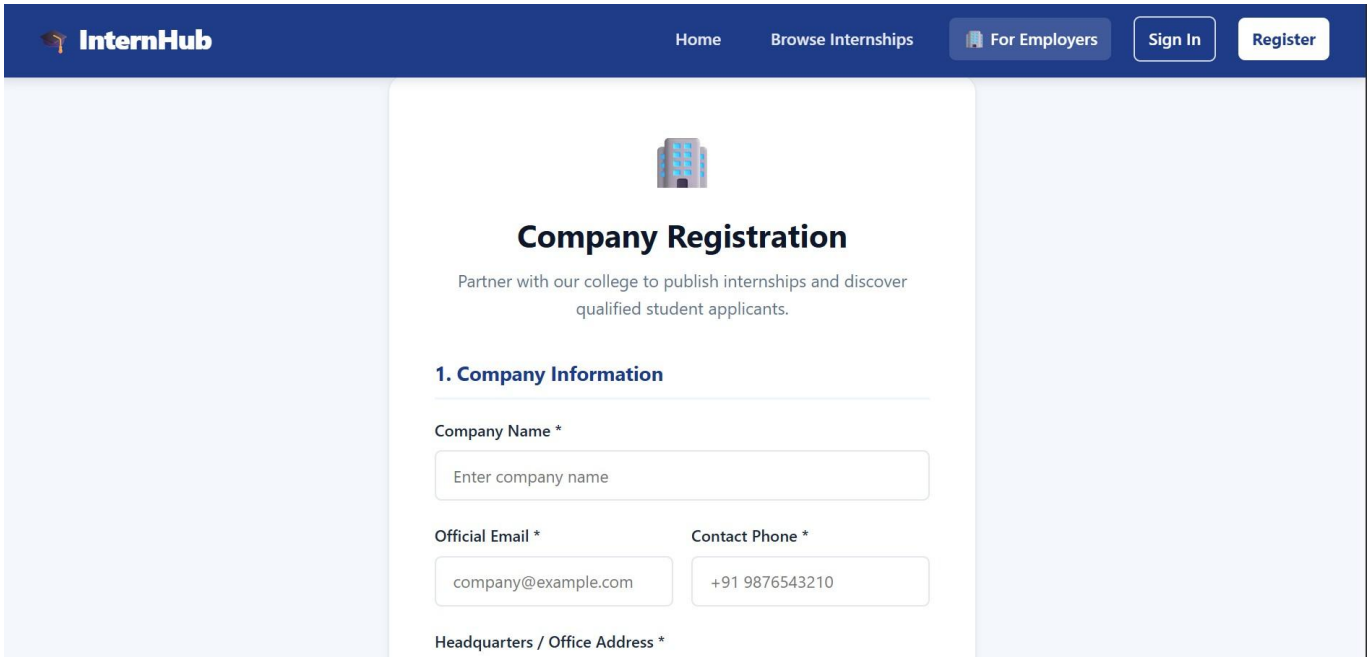
User Registration page

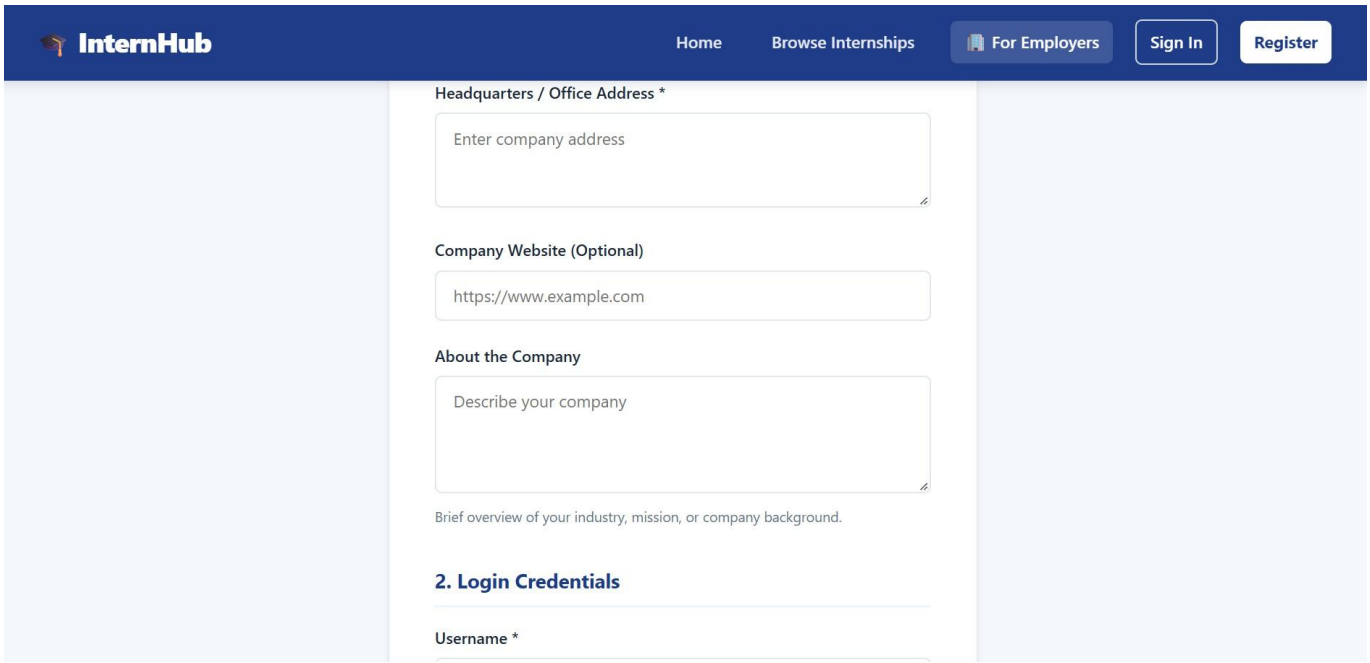


Login Page

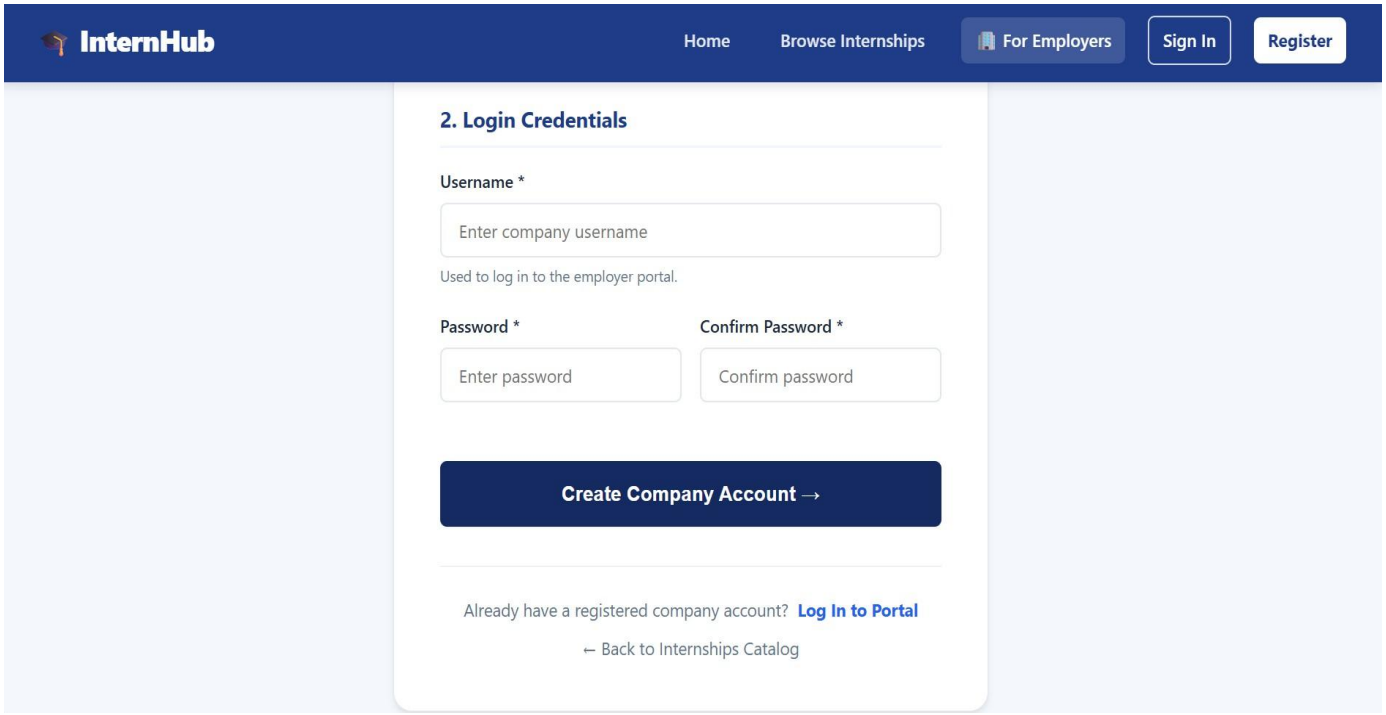


Company Registration Page

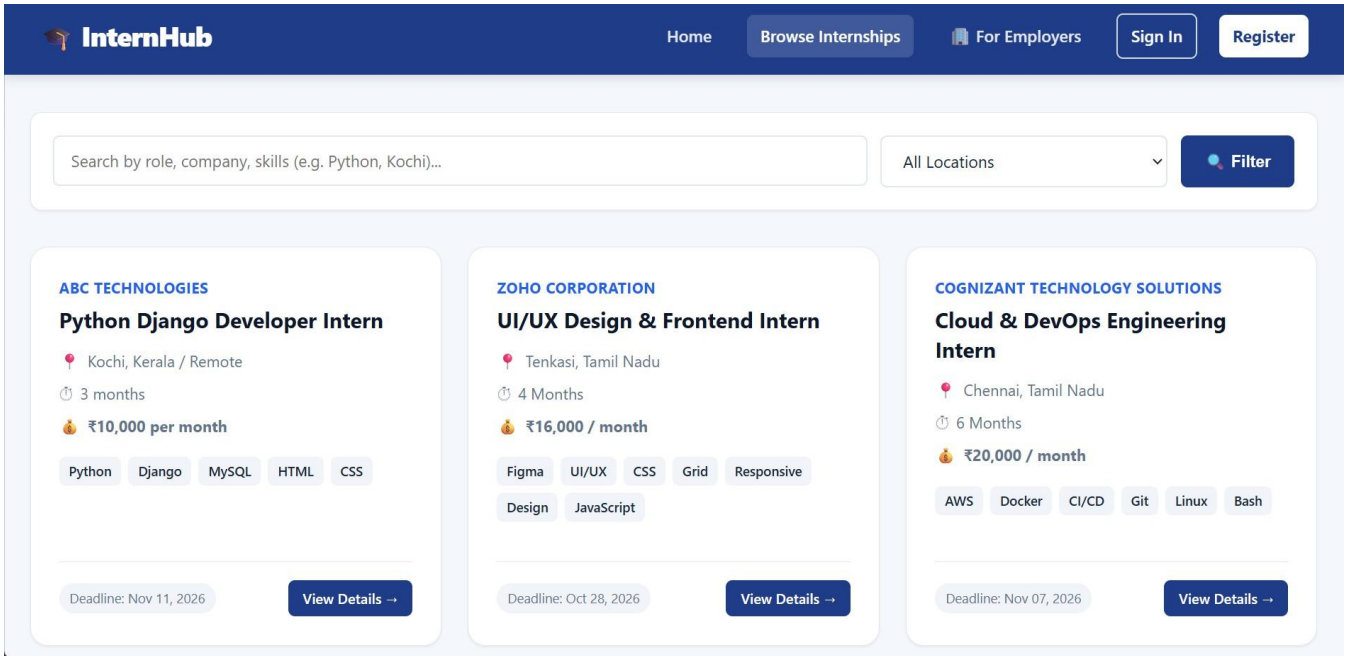




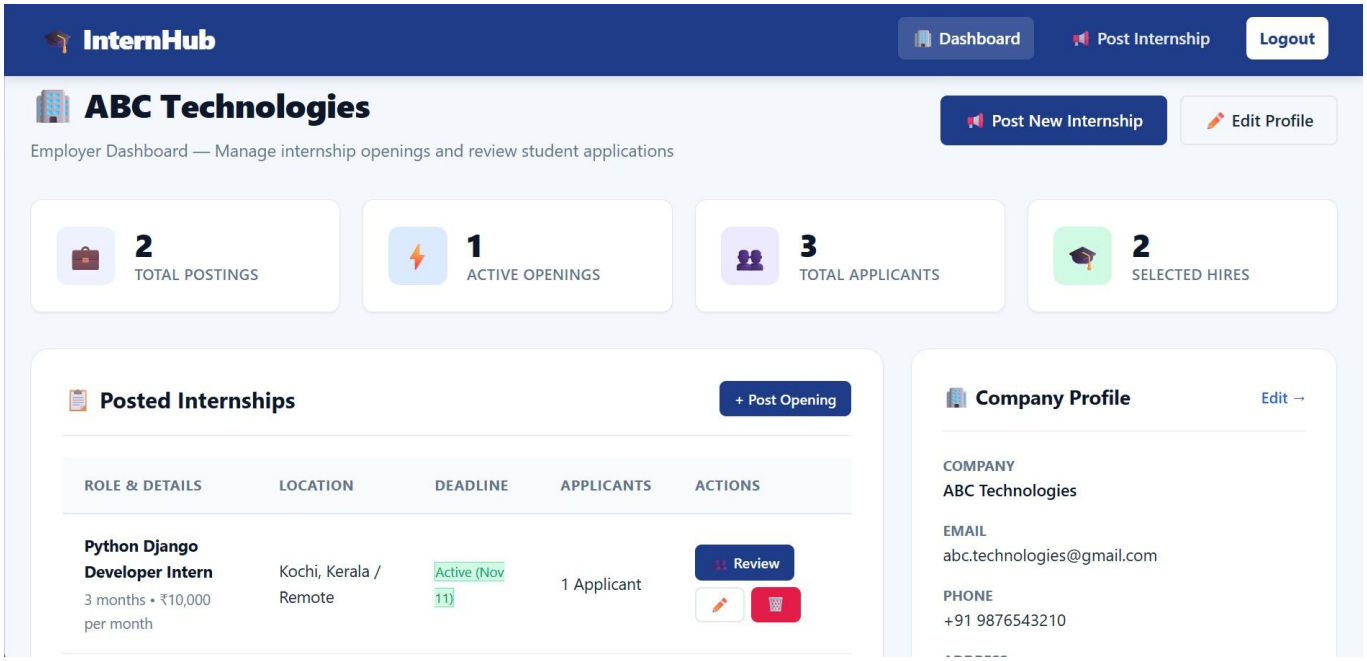
Company Registration Page



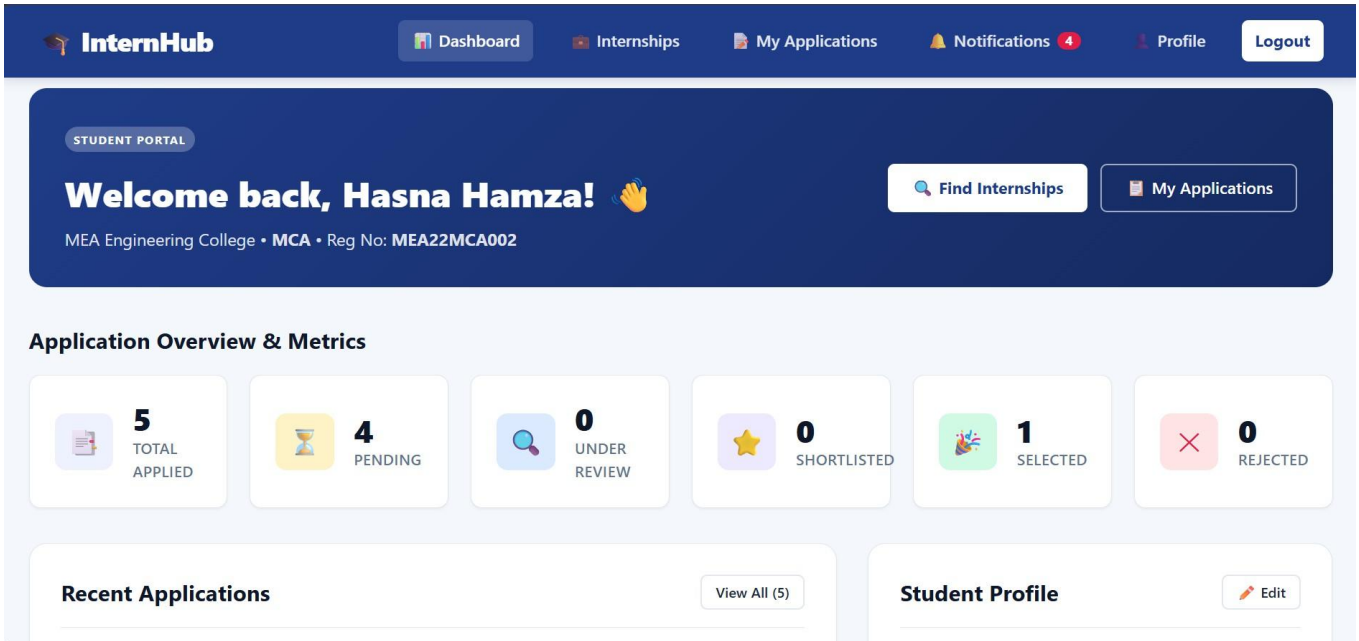
Internship Page



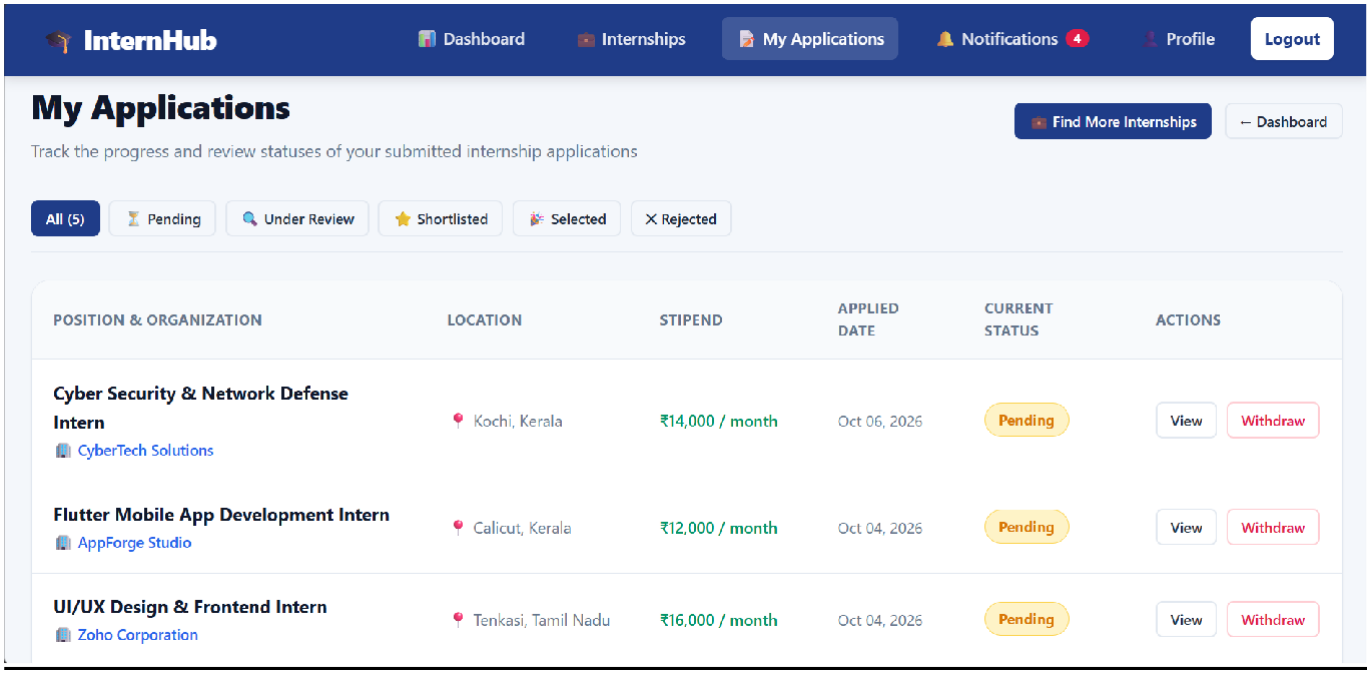
Company Dashboard



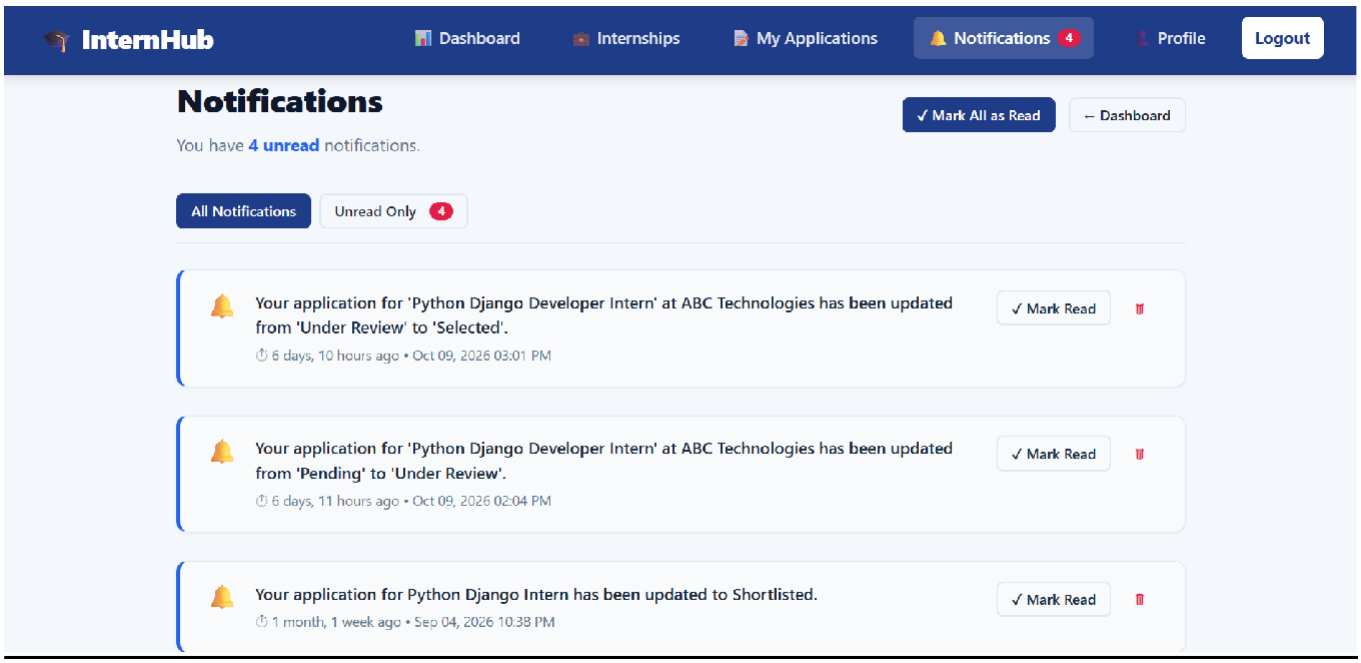
Student Dashboard



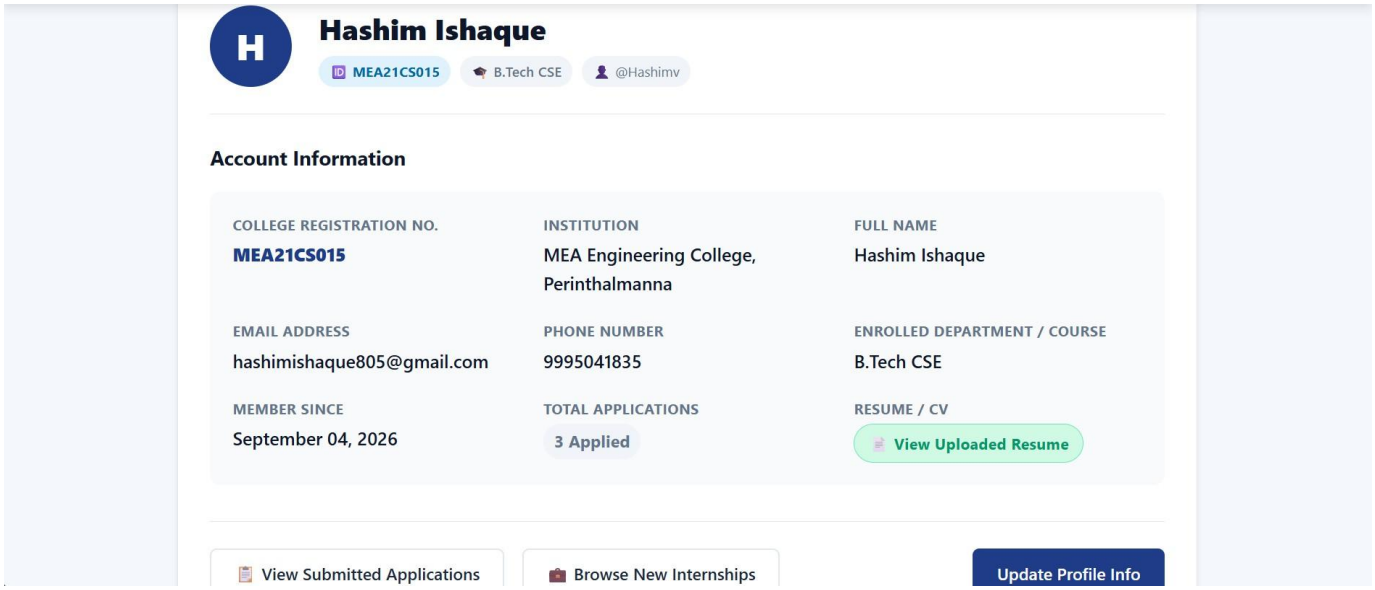
Applications Page



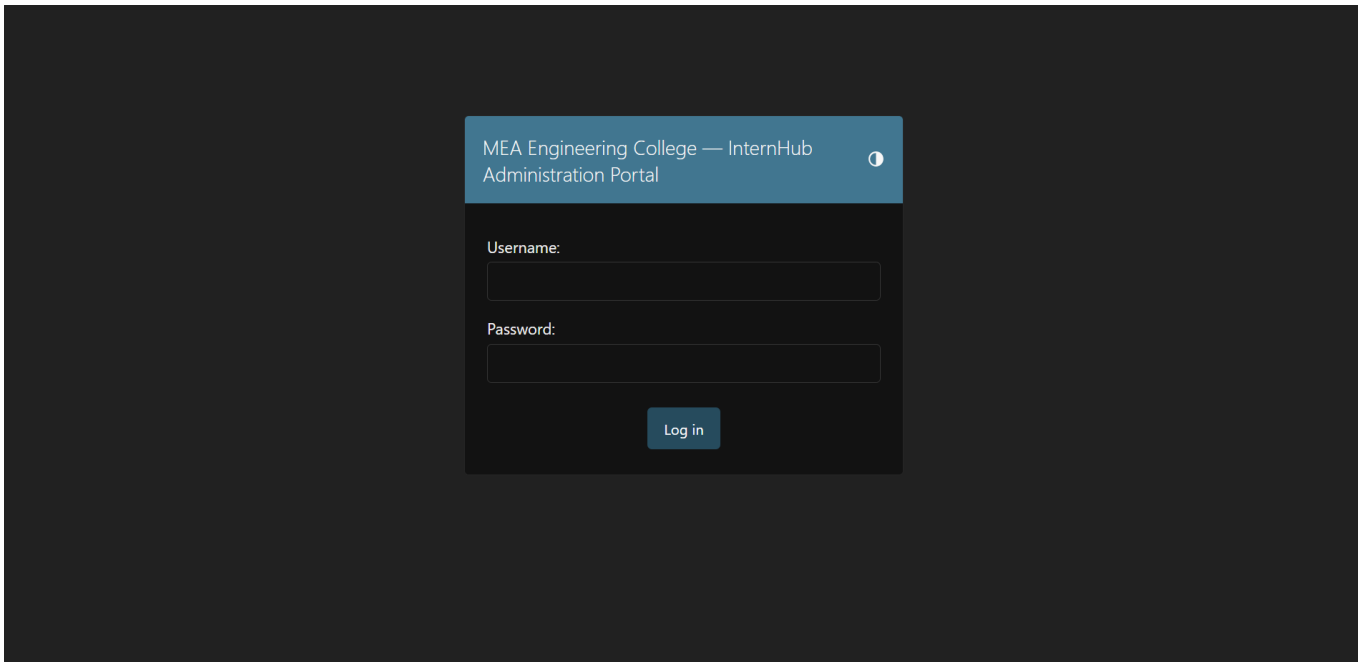
Notifications Page



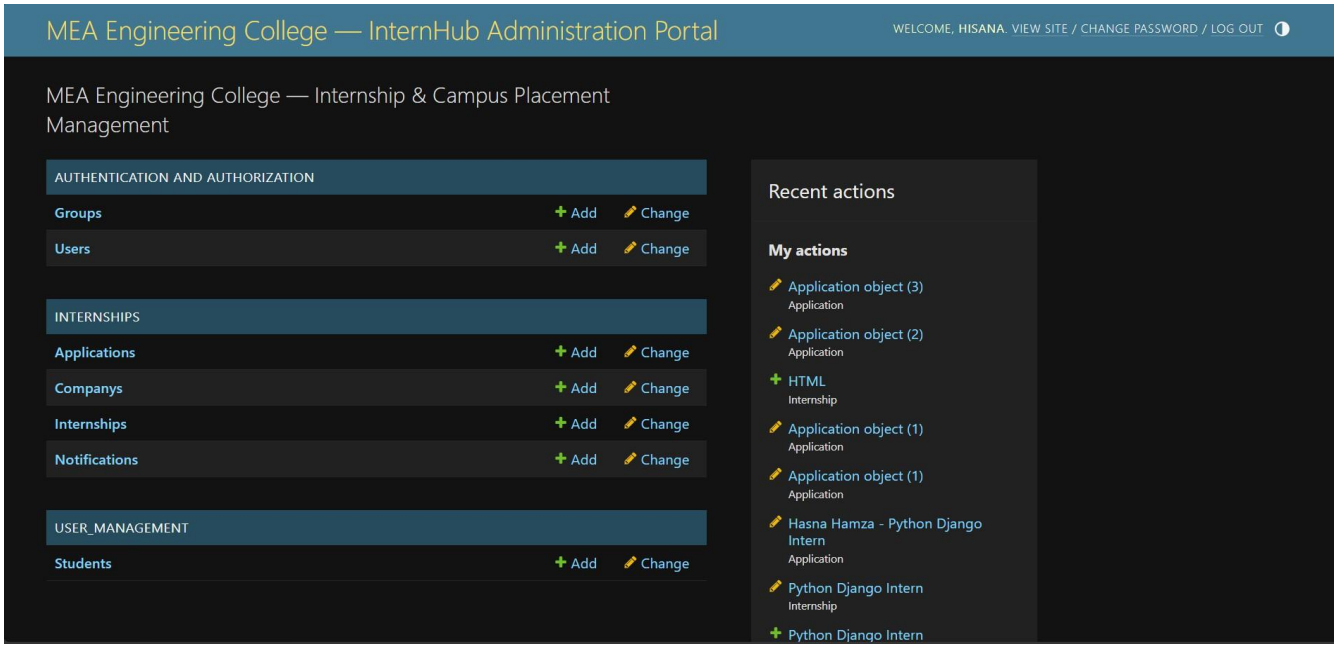
Student Profile Page



Admin Login Page



Admin Dashboard



BIBLIOGRAPHY

13. BIBLIOGRAPHY

Websites

[1] *https://docs.djangoproject.com/*
[2] *https://docs.python.org/3/*
[3] *https://dev.mysql.com/doc/*
[4] *https://developer.mozilla.org/en-US/*
[5] *https://getbootstrap.com/docs/*

Books

[1] *Django for Beginners – William S. Vincent*
[2] *Two Scoops of Django – Daniel Feldroy*
[3] *Learning SQL – Alan Beaulieu*
[4] *Python Crash Course – Eric M*
[5] *Web Development with Django*